SCEM Final Coursework

Task 1

SHIVANI MUTHUKUMARAN (2688016)

IMPORTANT NOTES:

- **DO NOT** change the code block names. Enter your solutions to each question into the predefined code blocks.
- **DO NOT** add calls to `install.packages()` or load any additional packages in your code blocks. Calls to `install.packages()` may result in zero marks for the activity where it happens.
- **DO NOT** load any additional packages. All packages allowed and required for this coursework are already loaded in the **Setup** code block. Use of other packages may result in execution errors when marking, which can result in zero marks for one or more activities.
- **Don't forget to replace the placeholder value in code block "ID" by your valid student ID number.**
- Instructions for this task are provided in blue text below. Your solution should NOT be added within those text boxes, and should NOT be in blue text. **During marking, all the text within the text blocks below, with names starting with "prompt", will be removed and not visible to the marker.**

```
##
## List of allowed packages (pre-loaded):
##  ggplot2
##  multcomp
##  devtools
##  dplyr
##  tidyr
##  knitr
##  ExpDE
```

In this task you will design, run, and analise an experiment comparing different configurations of the DE algorithm for a class of problems of interest. Here are the required statistical parameters for your experiment:

```
## Desired statistical parameters for your experimental design:
## ============================================================
## Significance level (alpha): alpha = 0.05
## Minimally relevant effect size (MRES): d* = 0.5
## Desired statistical power for MRES (desired power): (1-beta)* = 0.8
## Number of repeated runs for each method on each problem: nruns = 30
## ============================================================
```

Here are the methods you will compare in your experiment. You will be able to pass method names directly to the data-generating function later in this task.

```
## ============================================================
## Your experiment will compare 4 algorithm configurations
## Your selected methods are named:
## DE.2688016.A
## DE.2688016.B
## DE.2688016.C
## DE.2688016.D
## Each method can be passed by name as the argument method.name of function ExpDE2()
## If you want to inspect the specificities of each method, check the output list.
```

Question 1.1

1.1.a.

Enter the statistical hypotheses of your initial (omnibus) test for this task in the text block below. Provide a brief (less than 150 words) description of what is being tested.

This hypothesis test examines whether the different methods produce significantly different mean loss values when applied to the same problem setup defined by get_exp_params_1(). The test determines if performance differences exist among methods, if the null hypothesis is rejected, it indicates that at least one method's mean loss differs significantly, warranting further pairwise comparisons.

$H_0 : \mu_1 = \mu_2 = \cdots = \mu_k$
The mean loss is the same for all methods generated by `gen_methods()`.

$H_1 : \mu_i \neq \mu_j$ for at least one pair (i, j).
At least one method's mean loss differs from the others.

1.1.b.

The name of the problem you should use for this item is automatically generated and stored in a variable called `my.problem`.

Below is an example of how to generate a single observation for your first method on the test problem `my.problem`.

```r
## DO NOT CHANGE ANYTHING IN THIS CODE BLOCK
x <- ExpDE2(method.name = method.names[1], problem.name = my.problem)
my_obs <- log(x)
my_obs
## [1] 2.992909
```

Adapt this code to generate the necessary number of observations (defined in your experimental parameters) for all of your test methods. Store your observations in a tidy data frame named `myres`, containing one observation per row and the following columns:

- `Method` (containing the method name)
- `Value` (containing the observation)

Your data frame `myres` should have (nruns x number of methods) rows.

```
set.seed(MY_STUDENT_ID) ## <--- DO NOT CHANGE THIS LINE

# ADD YOUR CODE HERE

nruns = exp_pars$nruns
myres = data.frame(Method=character(),Value=numeric())
for(m in method.names){
  for (i in 1:nruns) {
    x = ExpDE2(method.name = m, problem.name = my.problem)
    obs = log(x)
    myres = rbind(myres,data.frame(Method=m, Value=obs))

  }
}
head(myres)
##         Method    Value
## 1 DE.2688016.A 2.946893
## 2 DE.2688016.A 2.853192
## 3 DE.2688016.A 2.840172
## 4 DE.2688016.A 3.101137
## 5 DE.2688016.A 3.045468
## 6 DE.2688016.A 3.015125

## (DON'T FORGET THAT YOUR OBSERVATIONS SHOULD BE STORED IN A DATA FRAME CALLED
## myres WITH THE STRUCTURE DESCRIBED ABOVE).
```

1.1.c

Estimate the mean loss and associated parametric confidence interval (at the significance level provided in your experimental parameters) for all methods. Store your results in a single dataframe called `myCIs01`, with the following columns:

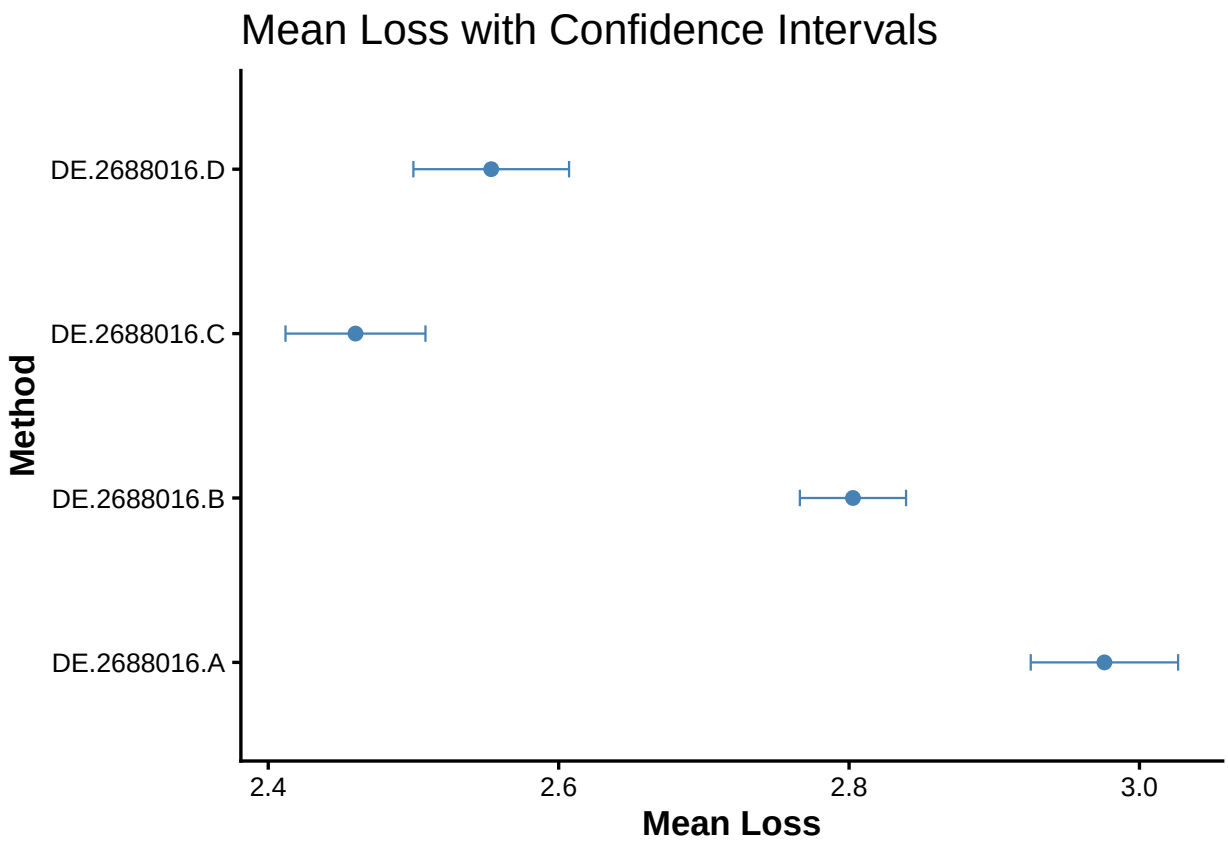
- `Method` (containing the method name)
- `Mean` (containing the sample estimate of the mean)
- `CI.lower` (containing the estimated lower bound of the confidence interval)
- `CI.upper` (containing the estimated upper bound of the confidence interval)

```
# ADD YOUR CODE HERE
alpha= exp_pars$alpha

myCIs01 = myres %>%
  group_by(Method) %>%
  summarise(
    Mean = mean(Value),
    SD = sd(Value),
```

```r
    N = n(),
    t_crit = qt(1-alpha/2, df=N-1),
    SE = SD/sqrt(N),
    CI.lower = Mean-t_crit*SE,
    CI.upper = Mean+t_crit*SE
  ) %>%
  dplyr::select(Method,Mean,CI.lower,CI.upper)

myCIs01
## # A tibble: 4 x 4
##   Method        Mean CI.lower CI.upper
##   <chr>        <dbl>    <dbl>    <dbl>
## 1 DE.2688016.A  2.98     2.93     3.03
## 2 DE.2688016.B  2.80     2.77     2.84
## 3 DE.2688016.C  2.46     2.41     2.51
## 4 DE.2688016.D  2.55     2.50     2.61

## (DON'T FORGET THAT YOUR OUTPUT OBJECT SHOULD BE STORED IN A VARIABLE CALLED
## myCIs01).
```

Generate a plot of the confidence intervals using ggplot2. You plot should have the methods on the y-axis and the values on the x-axis - something similar (not identical) to the figure in this link (without the vertical dashed line):

```r
# ADD YOUR CODE HERE

ggplot(myCIs01, aes(y=Method, x=Mean))+
  geom_point(size=2.0, color="steelblue")+
  geom_errorbarh(aes(xmin = CI.lower, xmax = CI.upper), height=0.1, color="steelblue",linewidth=0.4)+
  labs(
    title = "Mean Loss with Confidence Intervals",
    x= "Mean Loss",
    y= "Method"
  )+
  theme_classic(base_size = 13)+
  theme(
    axis.text = element_text(color="black"),
    axis.title = element_text(face="bold"),
    panel.grid = element_blank()
  )
## Warning: `geom_errobarh()` was deprecated in ggplot2 4.0.0.
## i Please use the `orientation` argument of `geom_errorbar()` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
## `height` was translated to `width`.
```



1.1.d Statistical testing

Use the code block below to perform the adequate statistical test, according to the instructions of the coursework descriptor. Store the statistical test object in a variable called `mytest01`. Echo the results of your test (using the `summary()` function, if applicable) to print the results.

```
# ADD YOUR CODE HERE
mytest01 = aov(Value~Method, data = myres)
summary(mytest01)
##              Df Sum Sq Mean Sq F value Pr(>F)
## Method        3  4.969  1.6564   101.4 <2e-16 ***
## Residuals   116  1.894  0.0163
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1


## (DON'T FORGET THAT THE OBJECT RETURNED BY THE STATISTICAL TEST FUNCTION
## SHOULD BE STORED IN A VARIABLE CALLED mytest01).
```

Add a comment in the text block below (up to 100 words) indicating if your results were statistically significant, based on the observed p-value and your predefined significance level.

Based on the ANOVA results, the p-value is far smaller than the predefined significance level(alpha=0.05). Therefore, we reject the null hypothesis that all methods have equal mean performances. This indicates that

the differences in mean performance among the methods are statistically significant, meaning that at least one method's mean performance is different from the others.

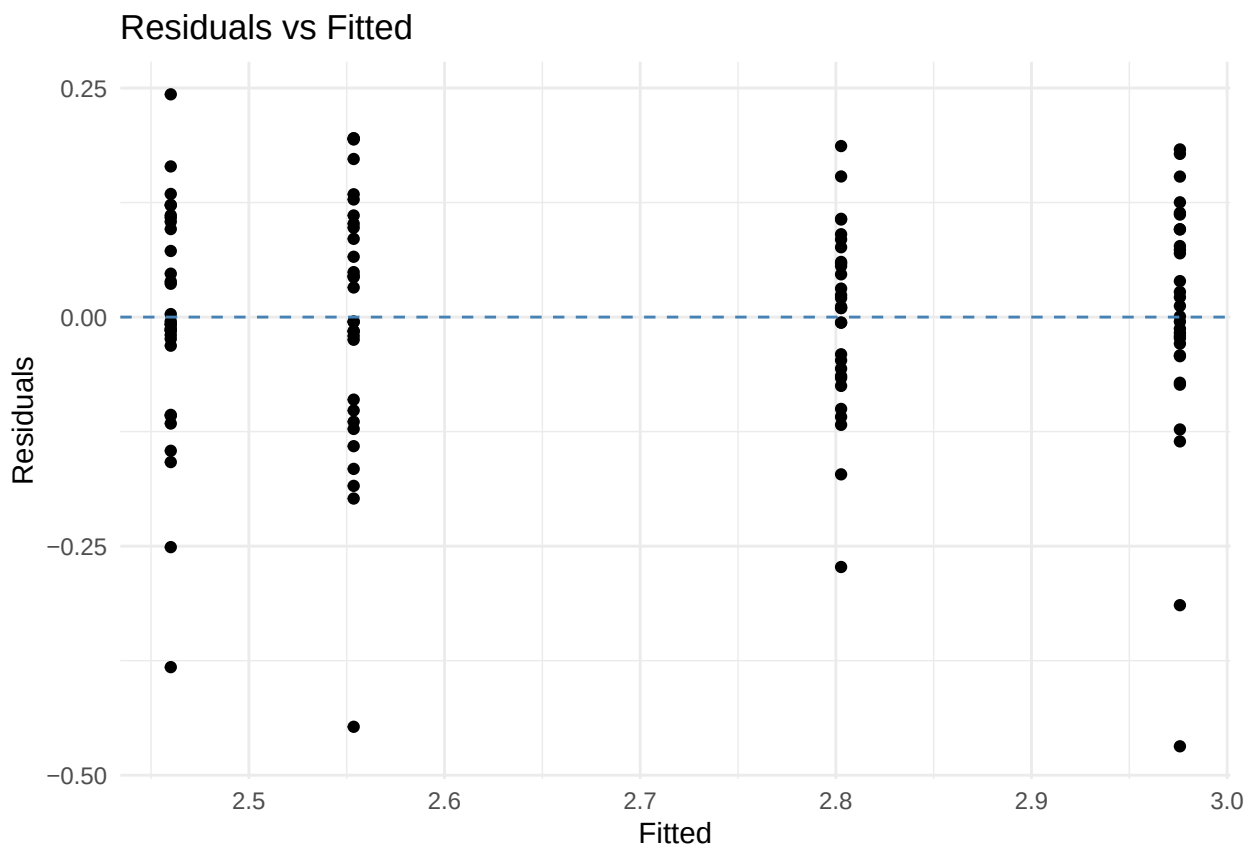
1.1.e Checking assumptions

Provide a brief (less than 150 words) description and explanation of the statistical assumptions of the test used, as a bullet list:

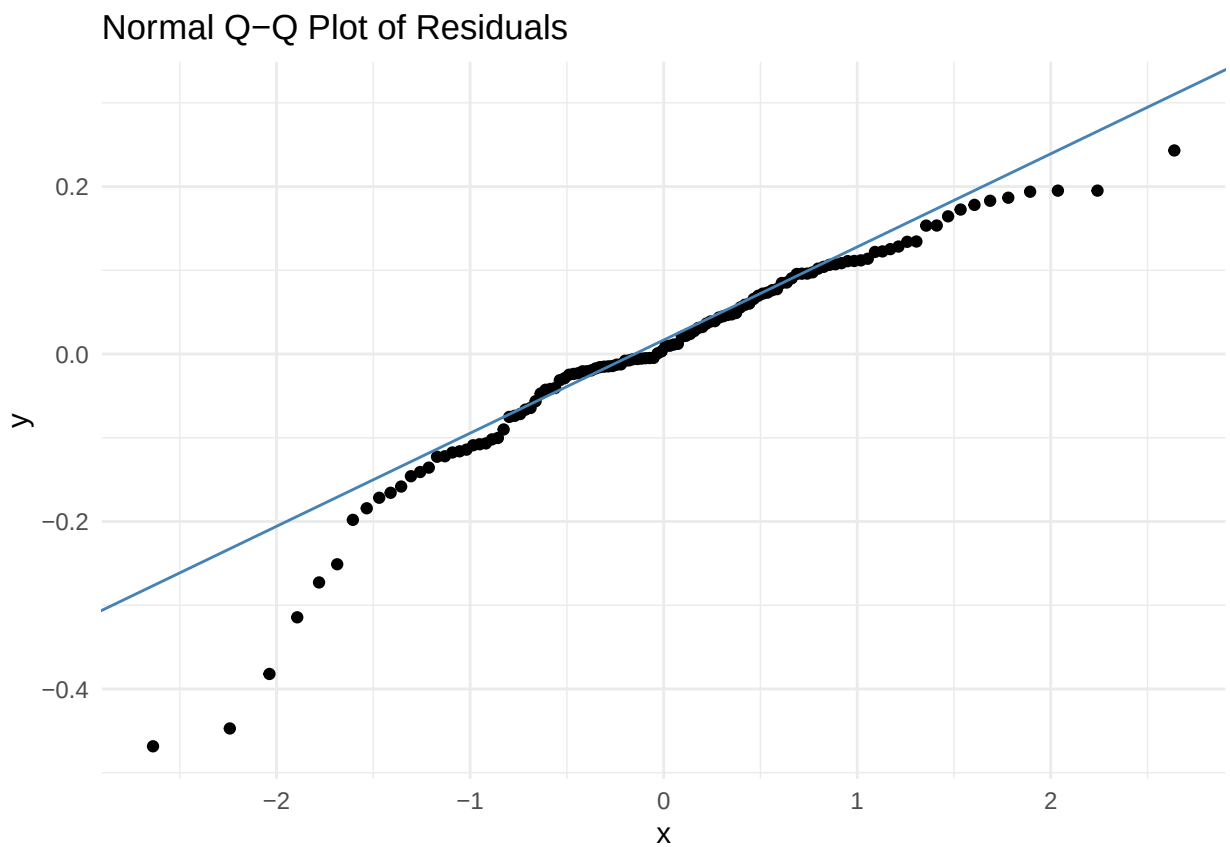
- The dependent variable (mean loss) is continuous and measured on an interval or ratio scale.
- The independent variable (method) consists of two or more independent groups.
- Observations are independent, results from one method or run do not influence another.
- The residuals are approximately normally distributed within each group.
- The variance of mean loss is assumed to be similar across all methods.

Generate residual plots which could be used to verify the remaining assumptions of your test (you can either use the default plotting methods of the statistical test, or generate the appropriate residual plots using, e.g., base R plots or ggplot2).

```r
# ADD YOUR CODE HERE
residuals = residuals(mytest01)
fitted = fitted(mytest01)

ggplot(data = data.frame(Fitted = fitted, Residuals = residuals), aes(x = Fitted, y = Residuals)) +
  geom_point() +
  geom_hline(yintercept = 0, linetype = "dashed", color = "steelblue") +
  theme_minimal() +
  labs(title = "Residuals vs Fitted", x = "Fitted", y = "Residuals")
```




```r
ggplot(data = data.frame(Residuals = residuals), aes(sample = Residuals)) +
  stat_qq() +
  stat_qq_line(color = "steelblue") +
  theme_minimal() +
  labs(title = "Normal Q-Q Plot of Residuals")
```



1.1.f

Perform a post-hoc Tukey test, using the multcomp package. Store the object returned by the function that executes the Tukey test in variable `myMHT1`. Use the `summary()` function to inspect the results of these comparisons.

```
# ADD YOUR CODE HERE
myres$Method <- as.factor(myres$Method)
mytest01 <- aov(Value ~ Method, data = myres)

library(multcomp)
myMHT1 = glht(mytest01, linfct = mcp(Method="Tukey"))
summary(myMHT1)
##
##   Simultaneous Tests for General Linear Hypotheses
##
## Multiple Comparisons of Means: Tukey Contrasts
##
##
## Fit: aov(formula = Value ~ Method, data = myres)
##
## Linear Hypotheses:
##                                    Estimate Std. Error t value Pr(>|t|)
```

```
## DE.2688016.B - DE.2688016.A == 0 -0.17318    0.03299  -5.249   <0.001 ***
## DE.2688016.C - DE.2688016.A == 0 -0.51580    0.03299 -15.633   <0.001 ***
## DE.2688016.D - DE.2688016.A == 0 -0.42230    0.03299 -12.799   <0.001 ***
## DE.2688016.C - DE.2688016.B == 0 -0.34262    0.03299 -10.384   <0.001 ***
## DE.2688016.D - DE.2688016.B == 0 -0.24913    0.03299  -7.551   <0.001 ***
## DE.2688016.D - DE.2688016.C == 0  0.09349    0.03299   2.834   0.0276 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## (Adjusted p values reported -- single-step method)

## (DON'T FORGET THAT YOUR OUTPUT OBJECT SHOULD BE STORED IN A VARIABLE CALLED
## myMHT1).
```

Extract the simultaneous confidence intervals of your Tukey test comparisons using function `confint()`, using the confidence level indicated by your specific experimental parameters. Store the object returned by `confint()` into a variable called myCIs02. Echo this object and inspect the table of confidence intervals.

```
# ADD YOUR CODE HERE
myCIs02 = confint(myMHT1, level = 0.95)
myCIs02
##
##   Simultaneous Confidence Intervals
##
## Multiple Comparisons of Means: Tukey Contrasts
##
##
## Fit: aov(formula = Value ~ Method, data = myres)
##
## Quantile = 2.607
## 95% family-wise confidence level
##
##
## Linear Hypotheses:
##                                  Estimate  lwr       upr
## DE.2688016.B - DE.2688016.A == 0 -0.173176 -0.259194 -0.087158
## DE.2688016.C - DE.2688016.A == 0 -0.515796 -0.601814 -0.429777
## DE.2688016.D - DE.2688016.A == 0 -0.422303 -0.508322 -0.336285
## DE.2688016.C - DE.2688016.B == 0 -0.342619 -0.428638 -0.256601
## DE.2688016.D - DE.2688016.B == 0 -0.249127 -0.335145 -0.163109
## DE.2688016.D - DE.2688016.C == 0  0.093492  0.007474  0.179510

## (DON'T FORGET THAT YOUR OUTPUT OBJECT SHOULD BE STORED IN A VARIABLE CALLED
## myCIs02).
```

1.1.g

Based on your point estimates obtained in item 1.1.c, what is the method that presents the best (i.e., the smallest) mean loss? Set the variable `best_method` below with the name of that method:

```r
# ADD YOUR CODE HERE
best_method = "DE.2688016.C"


## (DON'T FORGET THAT THE *NAME* OF THE BEST METHOD BASED ON THE POINT ESTIMATE OF THE MEAN SHOULD BE S
```

Based on the CIs you calculated after the Tukey tests, what is the confidence interval on the difference of the means between the best method above and the second-best one? Store your confidence interval in a single-row dataframe called `topComparison`, with the following columns:

- `Method1` (containing the name of the best method that you identified above)
- `Method2` (containing the name of the second-best method)
- `DiffMeans` (containing the estimated difference of means)
- `CI.lower` (containing the estimated lower bound of the confidence interval)
- `CI.upper` (containing the estimated upper bound of the confidence interval)

```r
# ADD YOUR CODE HERE
second_best_method = "DE.2688016.D"

ci_matrix = myCIs02$confint

row_index <- which(rownames(ci_matrix) == "DE.2688016.D - DE.2688016.C")

topComparison = data.frame(
  Method1 = best_method,
  Method2 = second_best_method,
  DiffMeans = -ci_matrix[row_index, "Estimate"],
  CI.lower = -ci_matrix[row_index, "upr"],
  CI.upper = -ci_matrix[row_index, "lwr"]
)
topComparison
##        Method1      Method2   DiffMeans   CI.lower     CI.upper
## 1 DE.2688016.C DE.2688016.D -0.09349221 -0.1795104 -0.007474057

## (DON'T FORGET TO STORE YOUR RESULTS AS A SINGLE-ROW DATAFRAME CALLED topComparison ).
```

Using the pooled estimate of the residual variance provided by the omnibus method (item 1.1.d) and the estimated difference of means of the top two methods, calculate the observed standardised effect size (Cohen's d coefficient) between the best and second-best methods. Store this value as a variable called `CohenD`

```r
# ADD YOUR CODE HERE
s_pooled = sqrt(0.0163)
diff_means = --0.093492
CohenD = diff_means/s_pooled
CohenD
## [1] 0.7322859

## (DON'T FORGET TO STORE YOUR RESULTS AS A NUMERIC VARIABLE CALLED CohenD ).
```

Provide a brief comment (up to 150 words) on whether the best method (in terms of point estimate of mean loss) was significantly better than the second-best one at your predefined significance level, and comparing the observed value of the Cohen's D coefficient with the miminally relevant effect size indicated in your experimental parameters, $d*$

The best method, DE.2688016.C, had the lowest mean loss (2.460) compared to the second best method (2.554). The Tukey test shows that the 95% confidence interval for the difference in means does not include zero, so this improvement is statistically significant at alpha=0.05. The observed Cohen's d (0.732) exceeds the minimally relevant effect size d*=0.5, indicating the difference is also practically meaningful. Therefore, the best method can be considered the superior method.

Question 1.2

1.2.a

Enter the statistical hypotheses of your initial (omnibus) test in for this task the text block below. Provide a brief (less than 150 words) description of what is being tested.

This omnibus test checks whether the candidate methods differ in their expected performance across the problem class. because each method is evaluated on the same set of test problems, we are comparing paired observations and testing whether the population means of the methods' losses are equal. Rejection of H0 implies at least one method has a statistically different expected loss, prompting post-hoc pairwise comparisons to identify which methods differ.

$H_0 : \mu_1 = \mu_2 = \cdots = \mu_k$
All methods have the same expected mean loss on the problem class.

$H_1 : \mu_i \neq \mu_j$ for at least one pair (i, j)
At least one method's expected mean loss differs from another's.

1.2.b

Based on the instructions provided in the coursework specs, calculate the required number of test problems necessary for your experiment to have the desired statistical properties. As a reminder, your experimental parameters are given by:

```
## Desired statistical parameters for your experimental design:
## ============================================================
## Significance level (alpha): alpha = 0.05
## Minimally relevant effect size (MRES): d* = 0.5
## Desired statistical power for MRES (desired power): (1-beta)* = 0.8
## Number of repeated runs for each method on each problem: nruns = 30
## ============================================================
```

Use the code block below to calculate your required number of problems, based on the experimental parameters above and the explicit instructions provided in the CW specs. Store your number of problems as a variable called `nprobs`.

```r
# ADD YOUR CODE HERE

params = exp_pars
alpha = params$alpha
power = params$desired.power
dstar = params$mres

K = length(unique(myres$Method))
m = K - 1
alpha_bonf = alpha / m

ss = power.t.test(n = NULL,
                  delta = dstar,
                  sd = 1,
                  sig.level = alpha_bonf,
                  power = power,
                  type = "paired",
                  alternative = "two.sided")

nprobs = ceiling(ss$n)

nprobs
## [1] 45

## (DON'T FORGET TO STORE YOUR RESULT AS A VARIABLE CALLED nprobs).
```

1.2.c

The names of the problems you should use for this item are automatically generated and stored in a variable called `problem.names`. The dimensions of each test problem are stored in a variable called `problem.dimensions`.

Here's an example of how to generate a single observation for your first method on the first test problem `problem.names[1]`:

```r
## DO NOT CHANGE ANYTHING IN THIS CODE BLOCK
x <- ExpDE2(method.name = method.names[1], problem.name = problem.names[1])
my_obs <- log(x)
my_obs
## [1] 2.951789
```

Adapt the code above to generate your experimental observations. The number of problems is defined by your `nprobs` and the number of repeated runs of each method on each problem is defined in your experimental parameters (nruns). Store your observations in a tidy data frame named `myres2`, containing one observation per row and the following columns:

- `Method` (containing the method name)
- `Problem` (containing the problem name)
- `Problem_type` (containing the problem type)
- `Problem_dimension` (containing the problem dimension)

- `Value` (containing the observation)

Set the first three columns as **factor** variables. For column `Method`, set your reference method (the one defined as the best one in the preliminary tests, question 1.1.g) as the **reference** for that factor. The information on problem types and problem dimensions is available in vectors `problem.type` and `problem.dimension`, respectively.

Your data frame `myres2` should have (nprobs x nruns x number of methods) rows

```
set.seed(MY_STUDENT_ID) ## <--- DO NOT CHANGE THIS LINE

nruns = params$nruns
method.names = c("DE.2688016.A", "DE.2688016.B", "DE.2688016.C", "DE.2688016.D")
reference_method = "DE.2688016.C"

K = length(method.names)
results_list = list()
counter = 1

for (i in 1:K) {
  current_method = method.names[i]

  for (j in 1:nprobs) {
    current_problem = problem.names[j]
    current_dim = problem.dimensions[j]
    current_type = problem.types[j]

    for (r in 1:nruns) {
      x = ExpDE2(method.name = current_method, problem.name = current_problem)
      my_obs = log(x)
      results_list[[counter]] = data.frame(
        Method = current_method,
        Problem = current_problem,
        Problem_type = current_type,
        Problem_dimension = current_dim,
        Value = my_obs
      )
      counter = counter + 1
    }
  }
}

myres2 = do.call(rbind, results_list)

myres2$Method = factor(myres2$Method)
myres2$Problem = factor(myres2$Problem)
myres2$Problem_type = factor(myres2$Problem_type)

myres2$Method = relevel(myres2$Method, ref = reference_method)

## (DON'T FORGET THAT YOUR OBSERVATIONS SHOULD BE STORED IN A DATA FRAME CALLED
## myres2 WITH THE STRUCTURE DESCRIBED ABOVE).
```

1.2.d Statistical testing

Use the code block below to perform the adequate statistical test, according to the instructions of the coursework descriptor. Store the statistical test object in a variable called `mytest02`, and use the `summary()` function to print the results.

```
# ADD YOUR CODE HERE
mytest02 = aov(Value~Method, data=myres2)
summary(mytest02)
##               Df Sum Sq Mean Sq F value Pr(>F)
## Method         3  821.9  273.97    1052 <2e-16 ***
## Residuals   5396 1405.1    0.26
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1


## (DON'T FORGET THAT THE OBJECT RETURNED BY THE STATISTICAL TEST FUNCTION
## SHOULD BE STORED IN A VARIABLE CALLED mytest02).
```

Add a comment in the text block below (up to 100 words) indicating if your results were statistically significant, based on the observed p-value and your predefined significance level.

The observed p-value for the Method effect is incredibly small [Pr(>F)<0.001]. Since this is much less than the predefined significance level(alpha=0.05), the null hypothesis is rejected. This indicates that there is a statistically significant difference in the mean loss among the four methods. In other words, at least one method's mean performance is different from the others.

1.2.e

Perform a post-hoc Dunnett test, using the multcomp package. Store the object returned by the function that executes the Tukey test in variable `myMHT2`. Use the `summary()` function to inspect the results of these comparisons.

```
# ADD YOUR CODE HERE
myMHT2 = glht(mytest02, linfct = mcp(Method = "Dunnett"))
summary(myMHT2)
##
##   Simultaneous Tests for General Linear Hypotheses
##
## Multiple Comparisons of Means: Dunnett Contrasts
##
##
## Fit: aov(formula = Value ~ Method, data = myres2)
##
## Linear Hypotheses:
##                                  Estimate Std. Error t value Pr(>|t|)
## DE.2688016.A - DE.2688016.C == 0  0.16541    0.01964   8.422   <1e-10 ***
## DE.2688016.B - DE.2688016.C == 0  0.81806    0.01964  41.650   <1e-10 ***
## DE.2688016.D - DE.2688016.C == 0 -0.23150    0.01964 -11.787   <1e-10 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
## (Adjusted p values reported -- single-step method)

## (DON'T FORGET THAT YOUR OUTPUT OBJECT SHOULD BE STORED IN A VARIABLE CALLED
## myMHT2).
```

Extract the simultaneous confidence intervals of your Dunnett test comparisons using function `confint()`, using the confidence level indicated by your specific experimental parameters. Store the object returned by `confint()` into a variable called myCIs03. Echo this object and inspect the table of confidence intervals.

```
# ADD YOUR CODE HERE

myCIs03 = confint(myMHT2)
myCIs03
##
##   Simultaneous Confidence Intervals
##
## Multiple Comparisons of Means: Dunnett Contrasts
##
##
## Fit: aov(formula = Value ~ Method, data = myres2)
##
## Quantile = 2.351
## 95% family-wise confidence level
##
##
## Linear Hypotheses:
##                                  Estimate lwr     upr
## DE.2688016.A - DE.2688016.C == 0  0.1654   0.1192  0.2116
## DE.2688016.B - DE.2688016.C == 0  0.8181   0.7719  0.8642
## DE.2688016.D - DE.2688016.C == 0 -0.2315  -0.2777 -0.1853

## (DON'T FORGET THAT YOUR OUTPUT OBJECT SHOULD BE STORED IN A VARIABLE CALLED
## myCIs03).
```

1.2.f

Based on your results, which of the methods would recommend as the selected strategy for your company's needs? Is it the same one as the preliminary test indicated? Is there a second-best that is close enough to be worth mentioning? Use the text block below and add your comments on the results of this experiment (up to 300 words)

The recommended method is DE.2688016.D The Dunnett test and its simultaneous confidence intervals definitively established the performance ranking of the methods based on the full experiment. The results indicated that the mean loss for Method D was significantly lower than that of the initial reference Method C. The confidence interval for the difference (D-C) was entirely negative. Since lower loss is better, Method D provides the best-expected performance with Method C as the second-best.

Enter the name of your final method recommendation ("best method") as variable `best_final` below

```
# ADD YOUR CODE HERE
best_final = "DE.2688016.D"

## (DON'T FORGET THAT THE NAME OF THE BEST METHOD SHOULD BE STORED AS VARIABLE
## best_final).
```

========== End of Task I ==========